DTSC 2302 Final Project
Jordan Beyer, Christopher Carroll, Andrew Farmer, Jordan Sentosa

Our topic is assessing environmental and socioeconomic factors to predict K-12 test proficiency across Charlotte/Mecklenburg neighborhoods. Charlotte-Mecklenburg represents one of the fastest-growing metro areas in the Southeast, yet this growth has not translated equally across its neighborhoods. The Quality of Life Explorer data makes this disparity visible in a way that aggregate county-level statistics obscure: neighborhoods a few miles apart can differ by 50 or more percentage points in test proficiency. Understanding what drives this gap matters not just academically, but also for how the county allocates resources. If proficiency outcomes are primarily school-specific, the policy response looks very different than if they are driven by the neighborhoods students go home to.

When looking at different NPA's (Neighborhood Profile Areas), we found that whether high or low, proficiency levels remained consistent across all grade levels within the same NPA. For example, if a specific NPA had lower proficiency for elementary, it was extremely likely that the middle and high school proficiencies were low as well. This was of interest because it implies that there are systemic and neighborhood-level factors impacting proficiency rather than only school-specific variables. We hypothesized that socioeconomic factors serve as the main drivers of these academic outcomes.

To test our hypothesis, we utilized both regression and classification models incorporating 21 unique predictors. These predictors include economic, housing and stability, digital and civic, demographics, and proximity variables. These predictors were chosen by selecting available data in 2023 tracked by the Charlotte/Mecklenburg Quality of Life Explorer, which could plausibly affect a child's ability to learn.

Our findings confirmed that socioeconomic factors were the strongest predictors of academic success, with access to food and nutrition services emerging as the primary driver and household income as a secondary driver. The Random Forest regression model achieved an R-squared of 0.85, and XGBoost was selected as the final classification model. This is a very short recount of our results. The remainder of this report details the analytical process behind these conclusions: first the regression model, including our approach, cross-validation results, and feature importance analysis; then the classification model, including how we categorized neighborhoods into four risk levels and why XGBoost was selected as the final model; and finally a conclusion summarizing our findings and offering policy recommendations aimed at closing the proficiency gap through targeted socioeconomic investment.

`Regression Model

Starting off with our regression model, our goal was to calculate the mean test proficiency percentage per neighborhood. We decided to create multiple models and put them into a 5-fold cross-validation method. To compare the models with each other, we mainly focused on the R-squared values, RMSE, and the feature importance spread. When including all of the features in our dataset, our Random Forest model performed the best with an .85 R-squared value (0.026 SD), and an RMSE of 8.63 (0.955 SD).

	Model	CV R2 Mean	CV R2 Std	CV RMSE Mean	CV RMSE Std
2	Random Forest	0.847118	0.028571	8.634672	0.955415
3	XGBoost	0.837038	0.027299	8.925854	0.913366
1	SVR	0.720158	0.041764	11.692886	0.958712
0	Linear Regression	0.649895	0.131501	12.869617	2.111106

To put these results in context, the K-12 test proficiency target has a mean of 47.22 and a standard deviation of 22.00, ranging from 7.73 to 95.33 across neighborhoods. An RMSE of 8.63, therefore, represents roughly 0.39 standard deviations of prediction error, which is reasonable given the breadth of the outcome range. Random Forest was selected as the final regression model over XGBoost, given its marginally higher CV R-squared (0.847 vs 0.837) and the consistency of its standard deviation across folds (0.029 vs 0.027), indicating similar stability with slightly better central performance. Taken together, the three modeling conditions tell a coherent story: food and nutrition services alone account for the majority of predictive signal, household income captures a meaningful secondary layer, and the remaining features collectively explain a modest but real additional portion of variance once both dominant predictors are removed.

Interestingly, when focusing on our Random Forest model, the feature importance was extremely skewed towards food and nutrition services – with an importance of 86% – meaning that food and nutrition services almost solely decided the predicted value, since the rest of the features had an importance of 2% or less. This makes food and nutrition services a very strong predictor of test proficiency in a certain area, but this model hides the relationship of the other variables with test proficiency, since one predictor overtakes the others by so much. Therefore, we decided to take out the food and nutrition services and rerun the models. Interestingly enough, when removed, our linear regression model slightly outperforms the random forest model (0.63 r-squared compared to 0.62 r-squared), and it had household income as its strongest predictor. When compared to models including food and nutrition, this is a sensible outcome, since in all other models besides random forest, household income was consistently a strong performer. Now, when we once again remove the best predictor, the features balance out, and the

best models return to being random forest and XGBoost. However, our R-squared values dropped once again, so for predicting purposes, it's best to include all features in our final model.

Afterwards, we looked into Pearson's correlation values since we had predictors that were strongly correlated in certain models. When calculating the value between food and nutrition services and test proficiency, the value was -0.735, indicating a strong negative relationship between the two. The value between household income and test proficiency was 0.782, which interestingly implies a stronger correlation, which was not exemplified in our models. However, when testing the correlation between the two, there is only a moderate relationship of +-0.665. While correlation between variables does not matter much due to our results indicating that XGBoost and Random Forest consistently performed the best, it's still important to note these relationships for a better understanding of our predictors.

Classification Model

Looking into our classification model, the goal was to evaluate the overall performance of a neighborhood by placing it into one of four categories: "High", "Moderate", "Low", and "At Risk". We determined each neighborhood's ranking based on its percentage of test proficiency, with a score of 0-25 being "At Risk", containing 68 neighborhoods, 25-50 being "Low", containing 214 neighborhoods, 50-75 being "Moderate", containing 94 neighborhoods, and 75-100 being "High", containing 79 neighborhoods.

Just like when crafting our regression model, we once again ran a 5-fold cross-validation method with multiple models to determine the best-performing model. Interestingly, similar to our regression results, the two best performing models were XGBoost and Random Forest, which both had the highest accuracy at around 0.70, with an SD of 0.036 and 0.053, respectively. Additionally, when looking at the feature importance, the results were once again similar to the regression model, with food and nutrition services being the most important and household income close behind in both models.

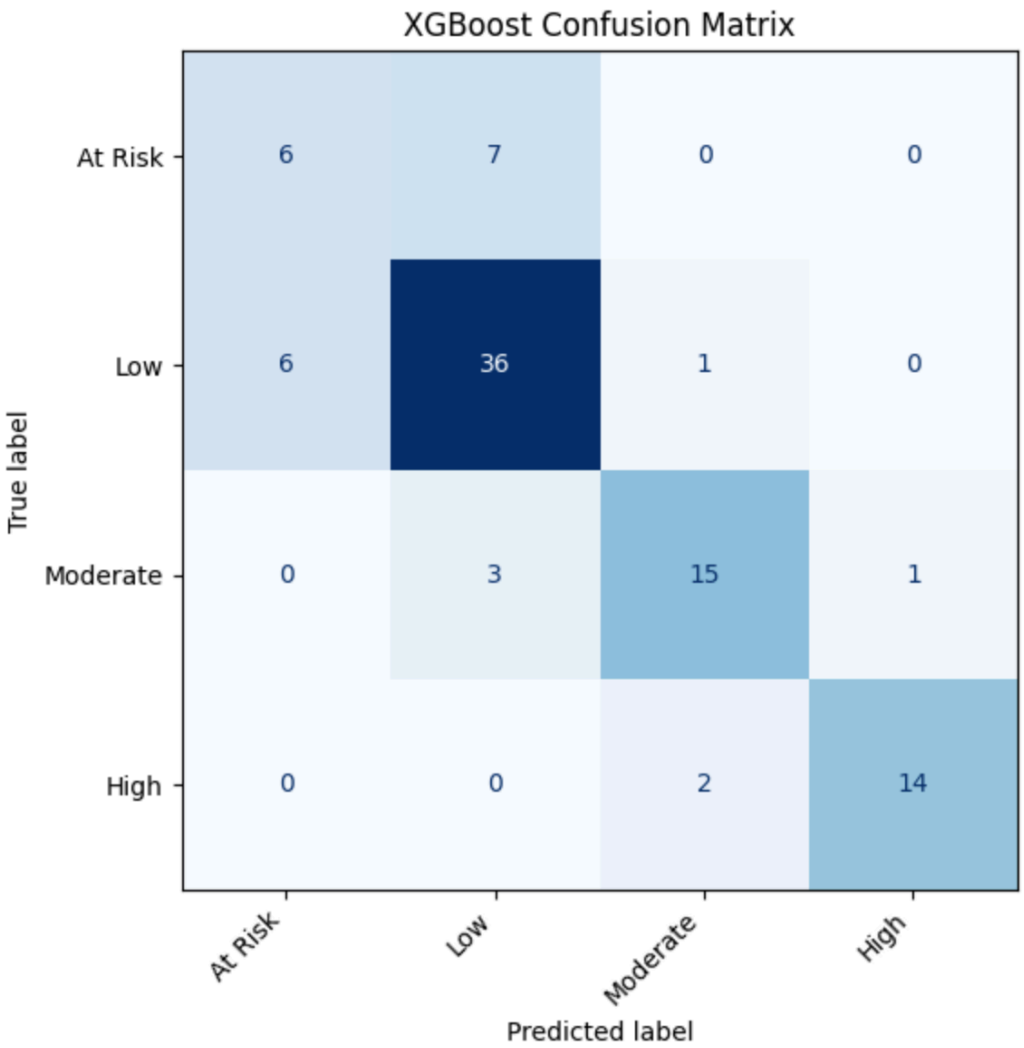
	Model	CV Accuracy Mean	CV Accuracy Std	CV Balanced Accuracy Mean	CV Balanced Accuracy Std	CV Macro F1 Mean	CV Macro F1 Std
4	XGBoost	0.697907	0.035969	0.660332	0.031082	0.661464	0.033389
3	Random Forest	0.700571	0.052842	0.619659	0.051937	0.631976	0.058611
0	LDA	0.642808	0.027982	0.579780	0.029745	0.586216	0.038523
2	SVM	0.579414	0.049006	0.582787	0.056503	0.553886	0.060270
1	QDA	0.587823	0.024456	0.523966	0.044650	0.511760	0.043972

Model	At Risk F1	Low F1	Moderate F1	High F1
LDA	0.524272	0.764873	0.503145	0.566372
QDA	0.420000	0.711111	0.356589	0.604317
SVM	0.533333	0.668919	0.479532	0.555556
Random Forest	0.390244	0.781250	0.582090	0.781250
XGBoost	0.490196	0.755682	0.606897	0.806202

Since the accuracy between both models was very close, we decided to run a partial dependency algorithm on both models, with the top 3 predictors for each being included, to

determine the dependency each model has on certain variables when predicting the outcome of a neighborhood. Interestingly, when the neighborhood is predicted to be at-risk, both models heavily relied on food and nutrition services as a variable, meaning that a neighborhood with a high usage of food and nutrition services almost definitely is in the “at-risk” zone. In the random forest model, household income had a positive trend with test proficiency, meaning that a higher household income made it more likely that the neighborhood would be educationally proficient. Interestingly, this relationship did not exist in the XGBoost model. Home Ownership also had a positive relationship in the Random Forest model, while XGBoost once again had no relationship present with its third-highest predictor, Housing Code Violations. This difference in interpretability favors Random Forest for understanding individual predictors, but interpretability alone is not sufficient grounds for model selection.

While Random Forest achieved higher precision on the At Risk class specifically (0.70 vs 0.50), XGBoost was selected as the final classification model based on its superior CV macro F1 mean (0.661 vs 0.632). Macro F1 averages performance equally across all four classes, making it the more appropriate metric here, given that our goal is neighborhood categorization across the full proficiency spectrum rather than optimizing detection of any single class. It is worth noting that XGBoost's lower At Risk precision reflects genuine ambiguity at the boundary between At Risk and Low neighborhoods, which the confusion matrix confirms: seven of thirteen true At Risk neighborhoods were predicted as Low, and six of forty-three true Low neighborhoods were predicted as At Risk. This boundary confusion is an honest reflection of the data rather than a model failure, and suggests that the distinction between the two lowest categories may be less categorical in practice than our binning implies.



Conclusion

We created regression and classification models, including XGBoost, to find factors that contributed to test proficiency percentage in Charlotte. In the regression models, we found that food and nutrition services were the largest predictors of test proficiency throughout the county. Areas that utilized these needs had percentages much lower on average, while areas that had less were much higher. This pattern also seemed to follow household income after we removed the food and services features. Food and services had a value of .86 while household income had a value of .782 in the separate model. When crafting our classification models, we separated our four quadrants into High, Moderate, Low, and At Risk. 68 neighborhoods were found to be at risk of having low test percentages, which were important in determining which factors signified low proficiency. Our best-performing models are Random Forest for regression and XGBoost for classification.

Based on these insights, increasing our funding for education in these areas that utilize the food and nutrition services more frequently can help improve our test proficiency scores. After-school programs and tutoring services can affect this in a positive way. Given that our

model showed a low food and nutrition score, which almost definitely placed a neighborhood in the "At Risk" category, addressing this need could have the single largest impact on improving educational outcomes. Alongside this, initiatives aimed at increasing household income, such as job training programs, workforce development, and economic investment in lower-income neighborhoods, could further improve test proficiency, given that income was the strongest predictor once food and nutrition services were removed from the model. Ultimately, our analysis demonstrates that improving K-12 test proficiency in Charlotte is not solely a matter of educational reform, but rather requires a broader investment in the socioeconomic well-being of the communities these students live in.

Beyond food access and income, the removal analysis surfaced a third layer of findings worth noting for policy. Once both dominant predictors were removed, rental costs emerged as the strongest remaining feature in both Random Forest and XGBoost, followed by high-speed internet access and home ownership. This suggests that housing stability and digital access represent meaningful secondary levers for improving educational outcomes, even after accounting for the more direct indicators of economic distress. Nuisance violations also appeared consistently, pointing toward the neighborhood's physical environment as a contributing factor. These findings suggest that code enforcement investment, rental assistance programs, and broadband expansion in lower-proficiency areas could complement the more direct food access and income interventions and merit further investigation as part of a comprehensive policy response. By addressing the root causes identified in our models, particularly food access and economic stability, policymakers and community leaders can begin to close the proficiency gap and create more equitable outcomes for students across all neighborhoods.

Proposal:
https://docs.google.com/document/d/1FJzEwnKGsRAc1B1Rbaj0ttaDpTLjBeBSZ43QV10l6dQ/edit?usp=sharing

Slides:
https://docs.google.com/presentation/d/18rdCv8lDlijVz3ORYVw8JlM5lwcvb8VuAU3Ydiq5sMM/edit?usp=sharing

Presentation:
https://drive.google.com/file/d/1hrWVDDiTTjy2KK562RnlKocShRnpQVQd/view?usp=sharing

GitHub:
https://github.com/jrdn-sentosa/charlotte-k12-proficiency

Database Usage:

```
CSV_FOLDER = "./features"
csv_files = glob.glob(os.path.join(CSV_FOLDER, "*.csv"))
conn = sqlite3.connect(":memory:")

for path in csv_files:
    file_name = os.path.splitext(os.path.basename(path))[0]
    df = pd.read_csv(path)[["NPA", "2023"]].rename(columns={"2023": file_name})
    df.to_sql(file_name, conn, index=False, if_exists="replace")

table_names = [os.path.splitext(os.path.basename(p))[0] for p in csv_files]
query = f'SELECT * FROM "{table_names[0]}"'

for table in table_names[1:]:
    query = f"""
    SELECT * FROM ({query.strip()})
    LEFT JOIN "{table}" USING(NPA)
    """

merged = pd.read_sql(query, conn)
conn.close()

merged.to_csv("merged_output.csv", index=False)
```

Regression Code:

In [59]:
```
import glob
import os
import sqlite3
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, StratifiedKFold, cross_validate, train_t
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from xgboost import XGBRegressor
import xgboost as xgb
```

In [ ]:
```
CSV_FOLDER = "./features"
csv_files = glob.glob(os.path.join(CSV_FOLDER, "*.csv"))
conn = sqlite3.connect(":memory:")

for path in csv_files:
    file_name = os.path.splitext(os.path.basename(path))[0]
    df = pd.read_csv(path)[["NPA", "2023"]].rename(columns={"2023": file_name})
    df.to_sql(file_name, conn, index=False, if_exists="replace")

table_names = [os.path.splitext(os.path.basename(p))[0] for p in csv_files]
query = f'SELECT * FROM "{table_names[0]}"'

for table in table_names[1:]:
    query = f"""
    SELECT * FROM ({query.strip()})
    LEFT JOIN "{table}" USING(NPA)
    """

merged = pd.read_sql(query, conn)
conn.close()

merged.to_csv("merged_output.csv", index=False)
```

Out[ ]:

	NPA	Bicycle Friendliness	Employment	Food and Nutrition Services	High Speed Internet Access	Home Ownership	Home Sales Price	Household Income
0	2	2	95.6%	23.4%	78.7%	40.3%	$488,364	$75,084
1	3	2	97.4%	2.9%	92.3%	39.2%	$667,092	$117,630
2	4	2	94.3%	0.3%	92.9%	100%	$1,493,043	$250,001
3	5	2	82.8%	49.7%	50.3%	14.9%	$255,031	$49,539
4	6	2	100%	39.3%	52.2%	37.3%	$445,698	$37,907

5 rows × 25 columns

In [61]:

```
df = pd.read_csv("merged_output.csv").drop(columns=["NPA"])

def clean_col(series):
    s = series.replace("--", np.nan)
    s = s.astype(str).str.replace(r"[$,%]", "", regex=True).str.replace(",", "")
    return pd.to_numeric(s, errors="coerce")

for col in df.columns:
    df[col] = clean_col(df[col])

target_cols = ["Test Proficiency - Elementary School", "Test Proficiency - Middle S
df = df.dropna(subset=target_cols, how="all")
df["Test Proficiency - K12"] = df[target_cols].mean(axis=1)

print(df["Test Proficiency - K12"].describe().round(2))
```

```
count    455.00
mean      47.22
std       22.00
min        7.73
25%       30.38
50%       40.20
75%       65.92
max       95.33
Name: Test Proficiency - K12, dtype: float64
```

In [62]:

```
feature_cols = [
    col for col in df.columns
    if col not in ["Test Proficiency - K12"] + target_cols
]
model_df = df[feature_cols + ["Test Proficiency - K12"]].dropna(subset=["Test Profi

X = model_df[feature_cols]
y = model_df["Test Proficiency - K12"]

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)
```

In [63]:
```
numeric_features = X.columns.tolist()

scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), numeric_features)
    ],
    remainder="drop",
)

tree_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ]), numeric_features)
    ],
    remainder="drop",
)
```

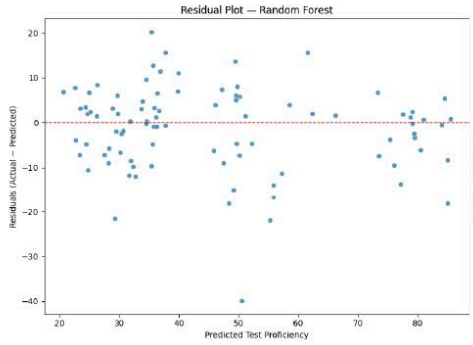
In [64]:
```
pipelines = {
    "Linear Regression": Pipeline([
        ("preprocessor", scaled_preprocessor),
        ("model", LinearRegression()),
    ]),
    "SVR": Pipeline([
        ("preprocessor", scaled_preprocessor),
        ("model", SVR(kernel="rbf", C=10.0, epsilon=0.1)),
    ]),
    "Random Forest": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", RandomForestRegressor(n_estimators=300, random_state=42)),
    ]),
    "XGBoost": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", XGBRegressor(
            objective="reg:squarederror",
            n_estimators=300,
            learning_rate=0.1,
            max_depth=4,
            subsample=0.9,
            colsample_bytree=0.9,
            random_state=42,
        )),
    ]),
}
```

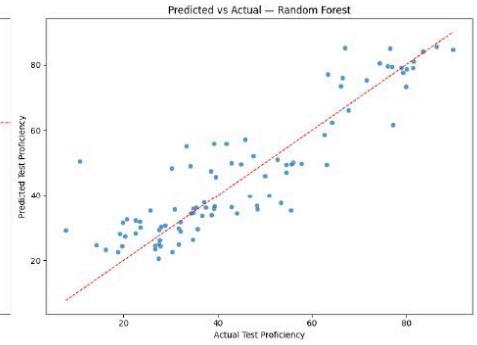
In [65]:
```
results = []
for name, pipeline in pipelines.items():
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
```

```
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    results.append({
        "Model": name,
        "R2": r2_score(y_test, y_pred),
        "RMSE": rmse,
        "MAE": mean_absolute_error(y_test, y_pred),
    })

results_df = pd.DataFrame(results).sort_values("R2", ascending=False)
display(results_df)
```

	Model	R2	RMSE	MAE
2	Random Forest	0.788660	9.423360	7.041374
3	XGBoost	0.787403	9.451346	7.278017
1	SVR	0.764941	9.938108	7.409165
0	Linear Regression	0.751072	10.227080	7.996898

In [66]:
```
best_name = results_df.iloc[0]["Model"]
best_pipeline = pipelines[best_name]
best_pred = best_pipeline.predict(X_test)
residuals = y_test - best_pred

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.scatterplot(x=best_pred, y=residuals, ax=axes[0], alpha=0.75)
axes[0].axhline(0, color="red", linestyle="--", linewidth=1)
axes[0].set_title(f"Residual Plot — {best_name}")
axes[0].set_xlabel("Predicted Test Proficiency")
axes[0].set_ylabel("Residuals (Actual - Predicted)")

sns.scatterplot(x=y_test, y=best_pred, ax=axes[1], alpha=0.75)
lims = [min(y_test.min(), best_pred.min()), max(y_test.max(), best_pred.max())]
axes[1].plot(lims, lims, color="red", linestyle="--", linewidth=1)
axes[1].set_title(f"Predicted vs Actual — {best_name}")
axes[1].set_xlabel("Actual Test Proficiency")
axes[1].set_ylabel("Predicted Test Proficiency")

plt.tight_layout()
plt.show()
```





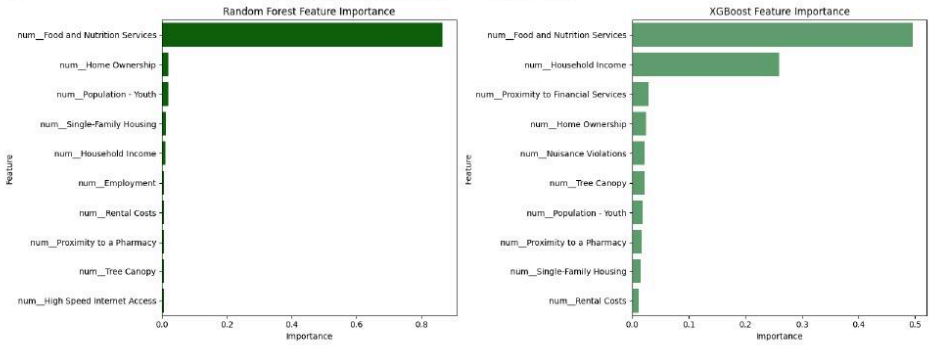
In [67]:
```
cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_results = []

for name, pipeline in pipelines.items():
    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring={
            "R2": "r2",
            "RMSE": "neg_root_mean_squared_error",
        },
        n_jobs=-1,
    )
    cv_results.append({
        "Model": name,
        "CV R2 Mean": scores["test_R2"].mean(),
        "CV R2 Std": scores["test_R2"].std(),
        "CV RMSE Mean": (-scores["test_RMSE"]).mean(),
        "CV RMSE Std": (-scores["test_RMSE"]).std()
    })

cv_results_df = pd.DataFrame(cv_results).sort_values("CV R2 Mean", ascending=False)
display(cv_results_df)
```

	Model	CV R2 Mean	CV R2 Std	CV RMSE Mean	CV RMSE Std
2	Random Forest	0.847118	0.028571	8.634672	0.955415
3	XGBoost	0.837038	0.027299	8.925854	0.913366
1	SVR	0.720158	0.041764	11.692886	0.958712
0	Linear Regression	0.649895	0.131501	12.869617	2.111106

In [68]:
```
def get_feature_importance(model_name):
    pipeline = pipelines[model_name]
    feature_names = pipeline.named_steps["preprocessor"].get_feature_names_out()
    importances  = pipeline.named_steps["model"].feature_importances_
    return pd.DataFrame({
```

```
        "Feature": feature_names,
        "Importance": importances,
    }).sort_values("Importance", ascending=False)

rf_importance  = get_feature_importance("Random Forest")
xgb_importance = get_feature_importance("XGBoost")

print(rf_importance.head(10))
print(xgb_importance.head(10))

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=False)
sns.barplot(data=rf_importance.head(10), x="Importance", y="Feature", ax=axes[0], c
sns.barplot(data=xgb_importance.head(10), x="Importance", y="Feature", ax=axes[1],
axes[0].set_title("Random Forest Feature Importance")
axes[1].set_title("XGBoost Feature Importance")
plt.tight_layout()
plt.show()
```

```
                          Feature  Importance
2   num__Food and Nutrition Services    0.865560
4                num__Home Ownership    0.018688
11           num__Population - Youth    0.018577
19        num__Single-Family Housing    0.012994
6              num__Household Income    0.011188
1                    num__Employment    0.007270
18                num__Rental Costs    0.006717
13      num__Proximity to a Pharmacy    0.006697
20                 num__Tree Canopy    0.006500
3    num__High Speed Internet Access    0.006233
                              Feature  Importance
2       num__Food and Nutrition Services    0.495469
6                 num__Household Income    0.259909
15  num__Proximity to Financial Services    0.028848
4                    num__Home Ownership    0.024160
10              num__Nuisance Violations    0.022175
20                     num__Tree Canopy    0.021789
11               num__Population - Youth    0.018369
13          num__Proximity to a Pharmacy    0.017071
19            num__Single-Family Housing    0.014581
18                     num__Rental Costs    0.012318
```



In [69]:
```
feature_name = 'Food and Nutrition Services'
target_name = 'Test Proficiency - K12'
```

```
pearson_corr = model_df[feature_name].corr(model_df[target_name])
print(f"Pearson correlation ({feature_name} vs {target_name}): {pearson_corr:.3f}")

feature_name = 'Household Income'
pearson_corr = model_df[feature_name].corr(model_df[target_name])
print(f"Pearson correlation ({feature_name} vs {target_name}): {pearson_corr:.3f}")

cross1_name = 'Food and Nutrition Services'
cross2_name = 'Household Income'
pearson_corr = model_df[cross1_name].corr(model_df[cross2_name])
print(f"Pearson correlation ({cross1_name} vs {cross2_name}): {pearson_corr:.3f}")
```

```
Pearson correlation (Food and Nutrition Services vs Test Proficiency - K12): -0.735
Pearson correlation (Household Income vs Test Proficiency - K12): 0.782
Pearson correlation (Food and Nutrition Services vs Household Income): -0.665
```

In [70]:

```
drop1 = ["Food and Nutrition Services"]
X_train_w1 = X_train.drop(columns=drop1)
X_test_w1 = X_test.drop(columns=drop1)
feats_w1 = X_train_w1.columns.tolist()

scaled_pre_w1 = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), feats_w1)
])
tree_pre_w1 = ColumnTransformer([
    ("num", Pipeline([("imputer", SimpleImputer(strategy="median"))]), feats_w1)
])

y_bins = pd.qcut(y_train, q=5, labels=False, duplicates="drop")
cv_w1  = list(StratifiedKFold(n_splits=5, shuffle=True, random_state=42).split(X_tr

pipelines_w1 = {
    "Linear Regression": Pipeline([("preprocessor", scaled_pre_w1), ("model", Linea
    "SVR": Pipeline([("preprocessor", scaled_pre_w1), ("model", SVR())]),
    "Random Forest": Pipeline([("preprocessor", tree_pre_w1),   ("model", RandomFor
    "XGBoost": Pipeline([("preprocessor", tree_pre_w1),   ("model", xgb.XGBRegresso
}

rows_w1 = []
for name, pipe in pipelines_w1.items():
    s = cross_validate(pipe, X_train_w1, y_train, cv=cv_w1,
                       scoring={"R2": "r2", "RMSE": "neg_root_mean_squared_error"})
    rows_w1.append({
        "Model": name,
        "CV R2 Mean": s["test_R2"].mean(),
        "CV R2 Std": s["test_R2"].std(),
        "CV RMSE Mean": (-s["test_RMSE"]).mean(),
        "CV RMSE Std": (-s["test_RMSE"]).std(),
    })

results_w1 = pd.DataFrame(rows_w1).sort_values("CV R2 Mean", ascending=False)
print(results_w1)
```

```python
best_w1 = pipelines_w1[results_w1.iloc[0]["Model"]]
best_w1.fit(X_train_w1, y_train)
y_pred_w1 = best_w1.predict(X_test_w1)
print(f"Best model (no Food/Nutrition): {results_w1.iloc[0]['Model']}")
print(f"Test R2:   {r2_score(y_test, y_pred_w1):.3f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_w1)):.3f}")
```

```
               Model  CV R2 Mean  CV R2 Std  CV RMSE Mean  CV RMSE Std
0  Linear Regression    0.631666   0.046021     13.473537     0.691314
2      Random Forest    0.625842   0.045048     13.603901     0.968566
3            XGBoost    0.600025   0.042980     14.078432     1.060508
1                SVR    0.352871   0.051281     17.892269     0.671004
Best model (no Food/Nutrition): Linear Regression
Test R2:   0.684
Test RMSE: 11.527
```

In [71]:

```python
lr_w1 = pipelines_w1["Linear Regression"]
lr_w1.fit(X_train_w1, y_train)
lr_w1_pred = lr_w1.predict(X_test_w1)
coef_df_w1 = pd.DataFrame({
    "Feature": lr_w1.named_steps["preprocessor"].get_feature_names_out(),
    "Coefficient": lr_w1.named_steps["model"].coef_,
}).sort_values("Coefficient", key=lambda s: s.abs(), ascending=False)
print(f"\nLinear Regression (no Food/Nutrition)")
print(coef_df_w1.head(15))
```

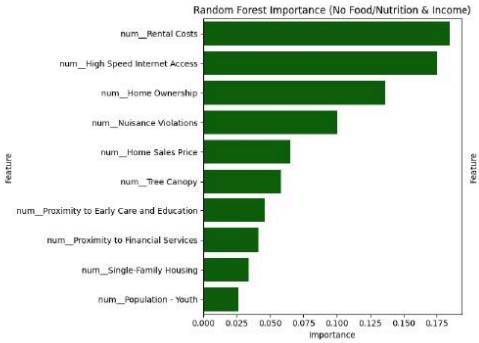
```
Linear Regression (no Food/Nutrition)
                                       Feature  Coefficient
5                        num__Household Income    10.487039
3                          num__Home Ownership     5.575918
14         num__Proximity to Financial Services     3.416337
19                              num__Tree Canopy     3.399836
18                  num__Single-Family Housing    -2.913066
13   num__Proximity to Early Care and Education    -1.831369
15  num__Proximity to Public Outdoor Recreation     1.818265
17                             num__Rental Costs     1.808158
9                      num__Nuisance Violations    -1.402915
1                               num__Employment     1.219128
0                    num__Bicycle Friendliness     0.897609
2               num__High Speed Internet Access     0.733784
12                 num__Proximity to a Pharmacy     0.708275
7                  num__Housing Code Violations    -0.678792
10                      num__Population - Youth     0.487227
```

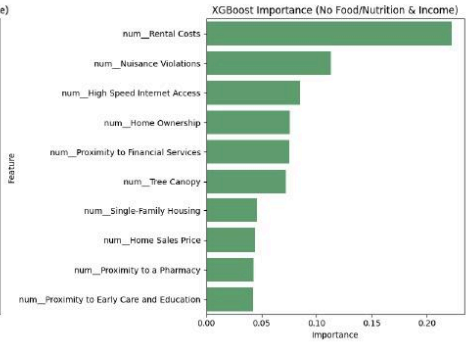
In [72]:

```python
drop2 = ["Food and Nutrition Services", "Household Income"]
X_train_w2 = X_train.drop(columns=drop2)
X_test_w2 = X_test.drop(columns=drop2)
feats_w2 = X_train_w2.columns.tolist()

scaled_pre_w2 = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), feats_w2)
])
tree_pre_w2 = ColumnTransformer([
    ("num", Pipeline([("imputer", SimpleImputer(strategy="median"))]), feats_w2)
```

```
])

y_bins2 = pd.qcut(y_train, q=5, labels=False, duplicates="drop")
cv_w2   = list(StratifiedKFold(n_splits=5, shuffle=True, random_state=42).split(X_t

pipelines_w2 = {
    "Linear Regression": Pipeline([("preprocessor", scaled_pre_w2), ("model", Linea
    "SVR": Pipeline([("preprocessor", scaled_pre_w2), ("model", SVR())]),
    "Random Forest": Pipeline([("preprocessor", tree_pre_w2),   ("model", RandomFor
    "XGBoost": Pipeline([("preprocessor", tree_pre_w2),   ("model", xgb.XGBRegressc
}

rows_w2 = []
for name, pipe in pipelines_w2.items():
    s = cross_validate(pipe, X_train_w2, y_train, cv=cv_w2,
                       scoring={"R2": "r2", "RMSE": "neg_root_mean_squared_error"})
    rows_w2.append({
        "Model": name,
        "CV R2 Mean": s["test_R2"].mean(),
        "CV R2 Std": s["test_R2"].std(),
        "CV RMSE Mean": (-s["test_RMSE"]).mean(),
        "CV RMSE Std": (-s["test_RMSE"]).std(),
    })

results_w2 = pd.DataFrame(rows_w2).sort_values("CV R2 Mean", ascending=False)
print(results_w2)

best_w2 = pipelines_w2[results_w2.iloc[0]["Model"]]
best_w2.fit(X_train_w2, y_train)
y_pred_w2 = best_w2.predict(X_test_w2)
print(f"Best model (no Food/Nutrition or Income): {results_w2.iloc[0]['Model']}")
print(f"Test R2: {r2_score(y_test, y_pred_w2):.3f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_w2)):.3f}")

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=False)
for ax, model_name, color in zip(axes, ["Random Forest", "XGBoost"], ["#026C02", "#
    pipe = pipelines_w2[model_name]
    pipe.fit(X_train_w2, y_train)
    imp_df = pd.DataFrame({
        "Feature": pipe.named_steps["preprocessor"].get_feature_names_out(),
        "Importance": pipe.named_steps["model"].feature_importances_,
    }).sort_values("Importance", ascending=False)
    sns.barplot(data=imp_df.head(10), x="Importance", y="Feature", ax=ax, color=col
    ax.set_title(f"{model_name} Importance (No Food/Nutrition & Income)")
plt.tight_layout()
plt.show()
```

```
              Model  CV R2 Mean  CV R2 Std  CV RMSE Mean  CV RMSE Std
2      Random Forest    0.566642   0.044335     14.638462     0.768743
0  Linear Regression    0.538911   0.066341     15.055437     0.845988
3            XGBoost    0.537766   0.061747     15.103980     0.947280
1                SVR    0.260553   0.052858     19.130789     0.681384
Best model (no Food/Nutrition or Income): Random Forest
Test R2: 0.620
Test RMSE: 12.642
```


Random Forest Importance (No Food/Nutrition & Income)
num__Rental Costs
num__High Speed Internet Access
num__Home Ownership
num__Nuisance Violations
num__Home Sales Price
num__Tree Canopy
num__Proximity to Early Care and Education
num__Proximity to Financial Services
num__Single-Family Housing
num__Population - Youth
Feature
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
Importance


XGBoost Importance (No Food/Nutrition & Income)
num__Rental Costs
num__Nuisance Violations
num__High Speed Internet Access
num__Home Ownership
num__Proximity to Financial Services
num__Tree Canopy
num__Single-Family Housing
num__Home Sales Price
num__Proximity to a Pharmacy
num__Proximity to Early Care and Education
Feature
0.00
0.05
0.10
0.15
0.20
Importance

Classification Code:

```
In [ ]: import glob
        import os
        import sqlite3
        import matplotlib.pyplot as plt
        import numpy as np
        import pandas as pd
        import seaborn as sns
        from sklearn.compose import ColumnTransformer
        from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDisc
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.impute import SimpleImputer
        from sklearn.inspection import PartialDependenceDisplay
        from sklearn.metrics import accuracy_score, balanced_accuracy_score, classification
        from sklearn.model_selection import StratifiedKFold, cross_val_predict, cross_valid
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import StandardScaler
        from sklearn.svm import SVC
        from xgboost import XGBClassifier
```

```
In [3]: CSV_FOLDER = "./features"
        csv_files = glob.glob(os.path.join(CSV_FOLDER, "*.csv"))
        conn = sqlite3.connect(":memory:")

        for path in csv_files:
            file_name = os.path.splitext(os.path.basename(path))[0]
            df = pd.read_csv(path)[["NPA", "2023"]].rename(columns={"2023": file_name})
            df.to_sql(file_name, conn, index=False, if_exists="replace")

        table_names = [os.path.splitext(os.path.basename(p))[0] for p in csv_files]
        query = f'SELECT * FROM "{table_names[0]}"'

        for table in table_names[1:]:
            query = f"""
            SELECT * FROM ({query.strip()})
            LEFT JOIN "{table}" USING(NPA)
            """

        merged = pd.read_sql(query, conn)
        conn.close()

        merged.to_csv("merged_output.csv", index=False)
```

```
In [4]: df = pd.read_csv("merged_output.csv").drop(columns=["NPA"])

        def clean_col(series):
            s = series.replace("--", np.nan)
            s = s.astype(str).str.replace(r"[$,%]", "", regex=True).str.replace(",", "")
            return pd.to_numeric(s, errors="coerce")

        for col in df.columns:
            df[col] = clean_col(df[col])

        target_cols = ["Test Proficiency - Elementary School", "Test Proficiency - Middle S
        df = df.dropna(subset=target_cols, how="all")
```

```
df["Test Proficiency - K12"] = df[target_cols].mean(axis=1)

print(df["Test Proficiency - K12"].describe().round(2))
```

```
count    455.00
mean      47.22
std       22.00
min        7.73
25%       30.38
50%       40.20
75%       65.92
max       95.33
Name: Test Proficiency - K12, dtype: float64
```

In [5]:

```
bins = [0, 25, 50, 75, 100]
labels = ["At Risk", "Low", "Moderate", "High"]
df["Proficiency Level"] = pd.cut(
    df["Test Proficiency - K12"],
    bins=bins,
    labels=labels,
    include_lowest=True,
)
print(df["Proficiency Level"].value_counts())
```

```
Proficiency Level
Low         214
Moderate     94
High         79
At Risk      68
Name: count, dtype: int64
```

In [6]:

```
feature_cols = [
    col for col in df.columns
    if col not in ["Test Proficiency - K12", "Proficiency Level"] + target_cols
]
model_df = df[feature_cols + ["Proficiency Level"]].dropna(subset=["Proficiency Lev

X = model_df[feature_cols]
y = model_df["Proficiency Level"].cat.codes
class_names = list(model_df["Proficiency Level"].cat.categories)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [7]:

```
numeric_features = X.columns.tolist()

scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), numeric_features)
    ],
    remainder="drop",
)
```

```
tree_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ]), numeric_features)
    ],
    remainder="drop",
)
```

In [8]:
```
pipelines = {
    "LDA": Pipeline([
        ("preprocessor", scaled_preprocessor),
        ("model", LinearDiscriminantAnalysis()),
    ]),
    "QDA": Pipeline([
        ("preprocessor", scaled_preprocessor),
        ("model", QuadraticDiscriminantAnalysis()),
    ]),
    "SVM": Pipeline([
        ("preprocessor", scaled_preprocessor),
        ("model", SVC(kernel="rbf", class_weight="balanced", random_state=42)),
    ]),
    "Random Forest": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", RandomForestClassifier(
            n_estimators=300, random_state=42, class_weight="balanced"
        )),
    ]),
    "XGBoost": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", XGBClassifier(
            objective="multi:softprob",
            num_class=4,
            eval_metric="mlogloss",
            n_estimators=300,
            learning_rate=0.1,
            max_depth=4,
            subsample=0.9,
            colsample_bytree=0.9,
            random_state=42,
        )),
    ]),
}
```

In [9]:
```
results = []
for name, pipeline in pipelines.items():
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Balanced Accuracy": balanced_accuracy_score(y_test, y_pred),
        "Macro F1": f1_score(y_test, y_pred, average="macro"),
    })
    print(f"\n{name}")
```

```
    print(classification_report(y_test, y_pred, target_names=class_names))

results_df = pd.DataFrame(results).sort_values("Macro F1", ascending=False)
display(results_df)
```

LDA

	precision	recall	f1-score	support
At Risk	0.53	0.62	0.57	13
Low	0.81	0.81	0.81	43
Moderate	0.67	0.63	0.65	19
High	0.73	0.69	0.71	16
accuracy			0.73	91
macro avg	0.69	0.69	0.69	91
weighted avg	0.73	0.73	0.73	91

QDA

	precision	recall	f1-score	support
At Risk	0.41	0.54	0.47	13
Low	0.76	0.67	0.72	43
Moderate	0.67	0.63	0.65	19
High	0.78	0.88	0.82	16
accuracy			0.68	91
macro avg	0.65	0.68	0.66	91
weighted avg	0.70	0.68	0.69	91

SVM

	precision	recall	f1-score	support
At Risk	0.42	0.62	0.50	13
Low	0.87	0.60	0.71	43
Moderate	0.52	0.68	0.59	19
High	0.65	0.69	0.67	16
accuracy			0.64	91
macro avg	0.61	0.65	0.62	91
weighted avg	0.69	0.64	0.65	91

Random Forest

	precision	recall	f1-score	support
At Risk	0.70	0.54	0.61	13
Low	0.80	0.91	0.85	43
Moderate	0.75	0.63	0.69	19
High	0.81	0.81	0.81	16
accuracy			0.78	91
macro avg	0.76	0.72	0.74	91
weighted avg	0.78	0.78	0.77	91

XGBoost

	precision	recall	f1-score	support
At Risk	0.50	0.46	0.48	13

```
         Low       0.78      0.84      0.81        43
    Moderate       0.83      0.79      0.81        19
        High       0.93      0.88      0.90        16

    accuracy                           0.78        91
   macro avg       0.76      0.74      0.75        91
weighted avg       0.78      0.78      0.78        91
```

	Model	Accuracy	Balanced Accuracy	Macro F1
4	XGBoost	0.780220	0.740805	0.750756
3	Random Forest	0.780220	0.722379	0.738684
0	LDA	0.725275	0.687104	0.685927
1	QDA	0.681319	0.679865	0.663724
2	SVM	0.637363	0.647937	0.617476

In [10]:

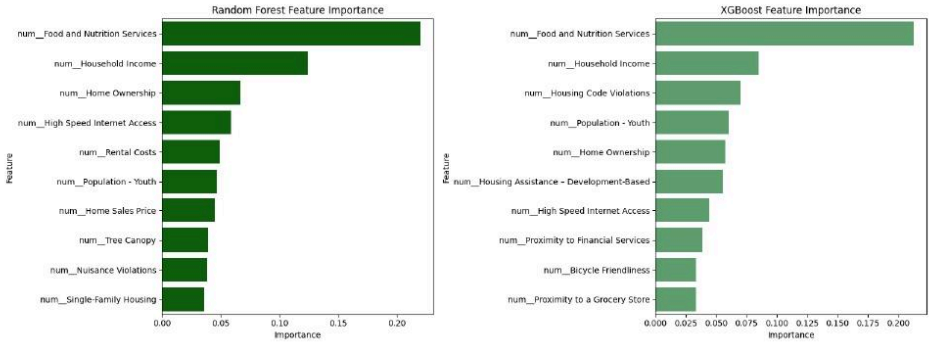
```python
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = []
cv_per_class_f1 = []

for name, pipeline in pipelines.items():
    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring={
            "Accuracy": "accuracy",
            "Balanced Accuracy": "balanced_accuracy",
            "Macro F1": "f1_macro",
        },
        n_jobs=-1,
    )
    y_cv_pred = cross_val_predict(pipeline, X_train, y_train, cv=cv, n_jobs=-1)
    report = classification_report(
        y_train,
        y_cv_pred,
        target_names=class_names,
        output_dict=True,
        zero_division=0,
    )
    cv_results.append({
        "Model": name,
        "CV Accuracy Mean": scores["test_Accuracy"].mean(),
        "CV Accuracy Std": scores["test_Accuracy"].std(),
        "CV Balanced Accuracy Mean": scores["test_Balanced Accuracy"].mean(),
        "CV Balanced Accuracy Std": scores["test_Balanced Accuracy"].std(),
        "CV Macro F1 Mean": scores["test_Macro F1"].mean(),
        "CV Macro F1 Std": scores["test_Macro F1"].std(),
    })
    cv_per_class_f1.append({
        "Model": name,
```

```
        **{f"{cls} F1": report[cls]["f1-score"] for cls in class_names},
    })

cv_results_df = pd.DataFrame(cv_results).sort_values("CV Macro F1 Mean", ascending=
cv_per_class_f1_df = pd.DataFrame(cv_per_class_f1).set_index("Model")
display(cv_results_df)
display(cv_per_class_f1_df)
```

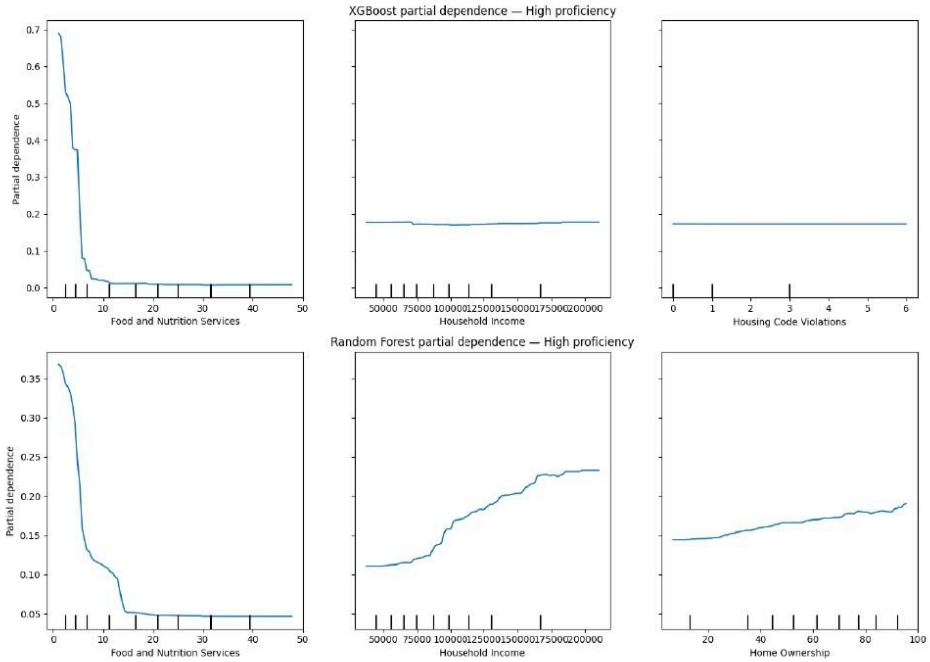
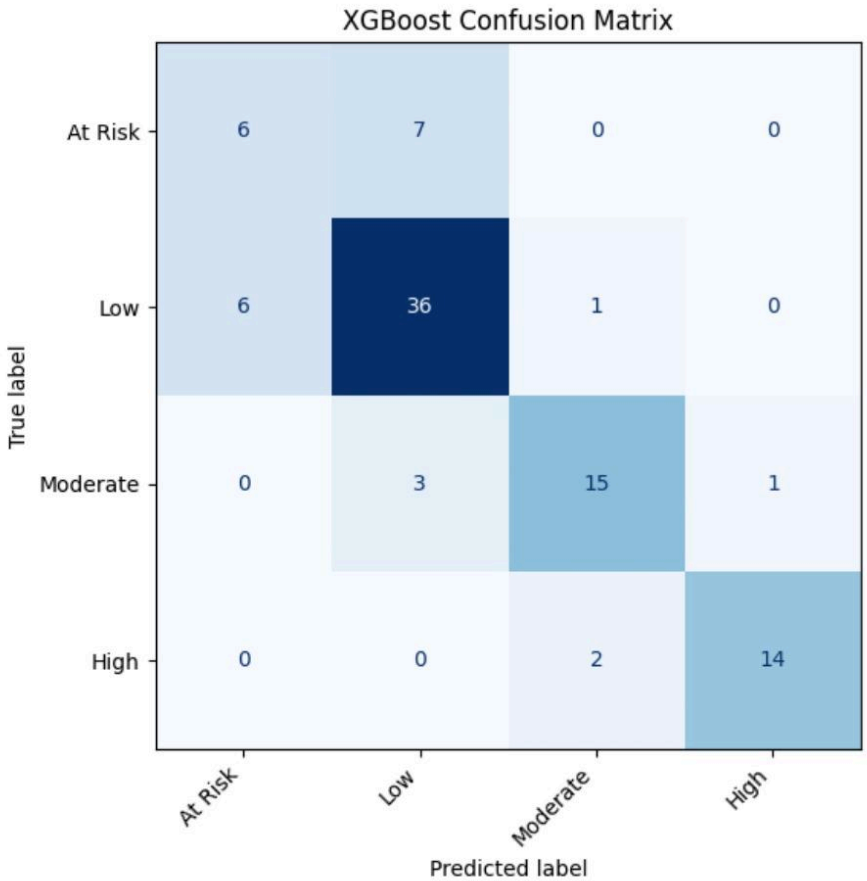
	Model	CV Accuracy Mean	CV Accuracy Std	CV Balanced Accuracy Mean	CV Balanced Accuracy Std	CV Macro F1 Mean	CV Macro F1 Std
4	XGBoost	0.697907	0.035969	0.660332	0.031082	0.661464	0.033389
3	Random Forest	0.700571	0.052842	0.619659	0.051937	0.631976	0.058611
0	LDA	0.642808	0.027982	0.579780	0.029745	0.586216	0.038523
2	SVM	0.579414	0.049006	0.582787	0.056503	0.553886	0.060270
1	QDA	0.587823	0.024456	0.523966	0.044650	0.511760	0.043972

Model	At Risk F1	Low F1	Moderate F1	High F1
LDA	0.524272	0.764873	0.503145	0.566372
QDA	0.420000	0.711111	0.356589	0.604317
SVM	0.533333	0.668919	0.479532	0.555556
Random Forest	0.390244	0.781250	0.582090	0.781250
XGBoost	0.490196	0.755682	0.606897	0.806202

In [11]:
```
def get_feature_importance(model_name):
    pipeline = pipelines[model_name]
    feature_names = pipeline.named_steps["preprocessor"].get_feature_names_out()
    importances  = pipeline.named_steps["model"].feature_importances_
    return pd.DataFrame({
        "Feature": feature_names,
        "Importance": importances,
    }).sort_values("Importance", ascending=False)

rf_importance  = get_feature_importance("Random Forest")
xgb_importance = get_feature_importance("XGBoost")

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=False)
sns.barplot(data=rf_importance.head(10), x="Importance", y="Feature", ax=axes[0], c
sns.barplot(data=xgb_importance.head(10), x="Importance", y="Feature", ax=axes[1],
axes[0].set_title("Random Forest Feature Importance")
axes[1].set_title("XGBoost Feature Importance")
plt.tight_layout()
plt.show()
```



In [12]:
```
rf_top = [f.replace("num__", "") for f in rf_importance["Feature"].head(3)]
xgb_top = [f.replace("num__", "") for f in xgb_importance["Feature"].head(3)]

fig, axes = plt.subplots(2, 1, figsize=(14, 10))
PartialDependenceDisplay.from_estimator(
    pipelines["XGBoost"], X_train, features=xgb_top, target=3, kind="average", ax=a
)
axes[0].set_title("XGBoost partial dependence — High proficiency")
PartialDependenceDisplay.from_estimator(
    pipelines["Random Forest"], X_train, features=rf_top, target=3, kind="average",
)
axes[1].set_title("Random Forest partial dependence — High proficiency")
plt.tight_layout()
plt.show()
```



In [15]:
```
xgb_model = pipelines["XGBoost"]
y_pred_xgb = xgb_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred_xgb)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap="Blues", colorbar=False, values_format="d")
ax.set_title("XGBoost Confusion Matrix")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```